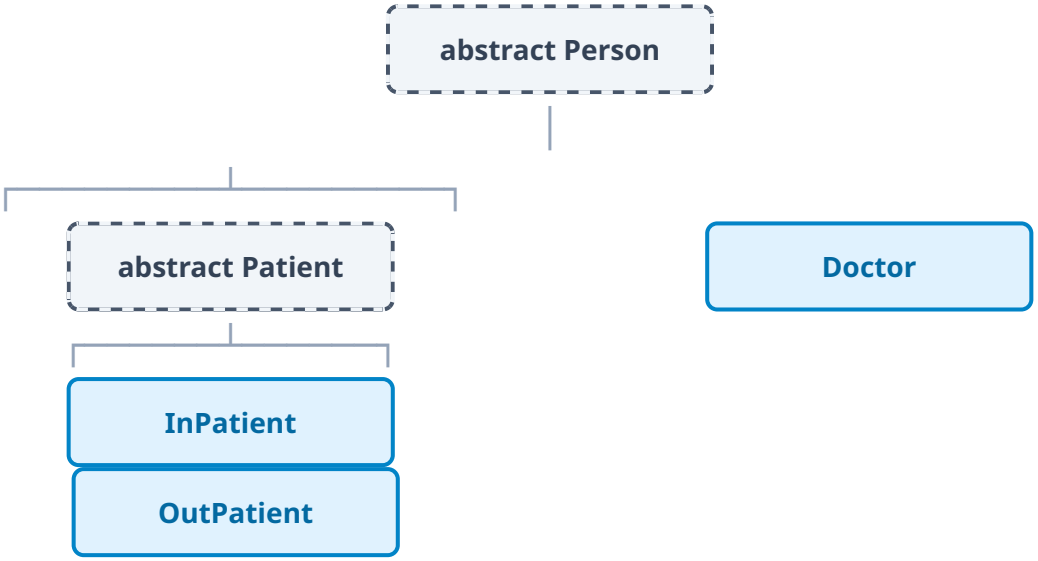
C# Abstract Classes & Inheritance Guide

Complete technical breakdown: Rules, member types, method defaults, and visual ordering

1. Visual Hierarchy & Solution Explorer Ordering

In Visual Studio, Solution Explorer sorts files strictly **alphabetically**. To make your project explorer visually reflect an inheritance hierarchy like your diagram:



Recommended Folder Structure:

```
Hospital Patient System/
├── Person/
│   ├── Person.cs            <- Abstract Base Class
│   ├── Doctor.cs            <- Derived Class
│   └── Patients/
│       ├── Patient.cs       <- Abstract Patient Base
│       └── Types/
│           ├── InPatient.cs  <- Derived Concrete Class
│           └── OutPatient.cs <- Derived Concrete Class
└── Program.cs
```

2. Inheriting from an Abstract Class

What You MUST Do:

- **Implement All Abstract Members:** You must provide a concrete body using the `override` keyword for every `abstract` method, property, indexer, or event defined in the base class. Failing to do so prevents compilation.
- **Chain Base Constructors:** If the abstract class defines constructors (especially parameterized ones), your child class must call them using `base(...)`.

What You ARE ABLE To Do:

- **Instantiation:** Unlike abstract classes, concrete classes can be created directly with `new Doctor()`.
- **Reuse Existing Logic:** Inherit and execute standard concrete methods or fields implemented in the abstract class without re-writing them.
- **Override Virtual Members:** Choose whether to override optional `virtual` methods or retain their default logic.
- **Add Custom Members:** Declare specialized fields, properties, or methods exclusive to the derived class.

What the Abstract Class CAN Do:

- **Partial Implementation:** Provide shared code alongside abstract definitions to prevent duplication across subclasses.
- **Enforce Architecture & Polymorphism:** Guarantee that all derived classes share common APIs, enabling lists like `List<Person>` to hold both `Doctor` and `Patient` instances.
- **Prevent Direct Object Creation:** Direct calls like `new Person()` trigger a compile error, guaranteeing that only complete child types exist.

3. Method Declarations: Abstract vs Virtual vs Default

> **Question: Is `public void fun();` inside an abstract class considered virtual or abstract by default?**
>
> **Answer: Neither!** It causes a **Compile Error (CS0501)**: *"Person.fun() must declare a body because it is not marked abstract, extern, or partial."*

In C#, methods inside an abstract class **have no implicit default type**. You must explicitly declare the behavior using keywords and method bodies:

Declaration Syntax	Keyword	Requires Body?	Must/Can Override in Child?
`public abstract void fun();`	`abstract`	**NO** (body prohibited)	**MUST** override in concrete child.
`public virtual void fun() { }`	`virtual`	**YES** (body required)	**CAN** optionally override.
`public void fun() { }`	None (Normal)	**YES** (body required)	**CANNOT** override (inherited as-is).

4. Code Example

```
public abstract class Person
{
    public string Name { get; set; }

    // 1. ABSTRACT METHOD: Must be overridden by child (No body)
    public abstract void Work();

    // 2. VIRTUAL METHOD: Can be overridden by child (Requires body)
    public virtual void Introduce()
    {
        Console.WriteLine($"Hi, I am {Name}.");
    }

    // 3. CONCRETE METHOD: Direct functionality (Requires body)
    public void Sleep()
    {
        Console.WriteLine("Sleeping...");
    }
}

public class Doctor : Person
{
    // MUST implement abstract method
    public override void Work()
    {
        Console.WriteLine("Treating patients...");
    }

    // OPTIONAL: Override virtual method
    public override void Introduce()
    {
        Console.WriteLine($"Hello, I am Dr. {Name}.");
    }
}
```